An Engineering Study of a Two-Tier QoS-Aware Scheduler for Single-Cell Private 5G

Anonymous Author(s)
Paper submitted for double-blind review

Abstract—A private 5G cell on a factory floor carries traffic whose classes want qualitatively different things: guaranteed-bitrate uplink video, motion control with millisecond deadlines, and elastic best-effort. Proportional fair (PF), the de facto default and the scheduler OpenAirInterface ships, represents neither a rate contract nor a deadline. Whether to replace it is an engineering decision, and this paper reports one made carefully. We build a two-tier scheduler—a network-utility-maximisation program setting per-flow rate targets on a ∼1 s horizon, and a per-slot drift-plus-penalty tracker realising them under a symbol-accurate PRB and PDCCH budget—and evaluate it against RR, PF and a class-aware gradient baseline.

We make no strong novelty claim for the design; it composes known parts, and we describe it because the composition is instructive. Our contribution is what building and measuring it carefully produced. First, an independent reproduction, in a simulator sharing no code with prior work, of two results the field has reported separately: configured grants dominate when the control channel binds (30/30 deadlines met against PF's 2/30), and deadline-blind scheduling fails silently (8/8 against 5/8, with PF's misses invisible in mean throughput). Second, and we think more useful: three modelling shortcuts that manufactured or destroyed results in our own evaluation, each quantified against its corrected version. In the worst case a headline finding—that our scheduler protected multi-flow uplink UEs where the baselines collapsed them—disappeared entirely once the uplink transport block was filled by the UE rather than the scheduler, as 3GPP requires. Third, an adoption decision with its reasoning: which regimes the design wins, which metric it loses on and why that is a metric choice rather than a failure, and an exact feasibility test the scheduler already computes that makes a graceful fallback possible without regime guessing. Simulator and scripts are released.

Index Terms—private 5G, MAC scheduling, quality of service, network utility maximisation, Lyapunov optimisation, configured grants, OpenAirInterface

I. Introduction

A private 5G network on a factory floor carries a workload that public mobile broadband does not. Mobile robots stream machine-vision and LIDAR uplink at a sustained rate; motion-control loops exchange small payloads under packet delay budgets (PDB) of a few milliseconds; firmware pushes and log uploads soak up whatever is left. These classes do not simply want more throughput—they want different things. A best-effort flow wants its share of the residue. A guaranteed-bitrate (GBR) flow wants a specific rate or nothing of value. A delay-critical flow wants its bytes before a deadline, after which they are worthless.

The scheduler that ships in OpenAirInterface (OAI), and the de facto default across LTE and NR, is proportional fair (PF) [1]. PF ranks users by instantaneous rate over smoothed average rate; its fixed point maximises $\sum_i \log R_i$ [2] but has no representation of a rate contract or a deadline.

A substantial literature adds QoS-awareness by modifying the PF metric with an urgency term—M-LWDF [3], EXP/PF and the Exp-rule [4], and the variants surveyed in [5]. A parallel thread casts allocation as network utility maximisation (NUM) [2] or as Lyapunov drift-plus-penalty control [6]–[8]. The question we ask is when is it worth for an operator to use a more sophisticated two level scheduler which might involve an LP or RL type solver, a control loop, and a non-trivial port into a real-time MAC. When is that worth paying for?

This paper answers that question for a private-5G factory target, where our two level scheduler uses a Linear Programming Formulation to determine rate allocations at a coarse time granularity and the MAC scheduler uses a simpler delay plus drift based formulation to meet those rate targets.

Contributions

We state the shape of this paper plainly, because it is not the usual one. The scheduler composes established parts—network utility maximisation [2] over a slow horizon feeding Lyapunov drift-plus-penalty [6] per slot, the pattern behind NVS [9] and the O-RAN RIC split [10]. We claim no novelty for it, and present it (Section IV) because the composition, and the places it goes wrong, are instructive. What we offer instead:

1) Independent reproduction of two results the field reports separately (Section VI), in a simulator sharing no code or abstractions with the work that first reported them. Configured grants dominate once the control channel binds—30/30 deadlines met against PF's 2/30—which Larrañaga et al. [11] report from ns-3/5G-LENA. And deadline-blind scheduling fails silently: PF meets 5/8 deadlines while posting an 86% mean that reads healthy on a dashboard. Reproduction is not novelty, but in a field where

most schedulers are evaluated once, in one simulator, by their authors, it is worth having.

2) Three modelling shortcuts that manufactured or destroyed results in our own evaluation (Section VII), each reported with the corrected measurement beside it. The sharpest: our scheduler appeared to protect multi-flow uplink UEs that the QoS-blind baselines collapsed to ∼3% of contract—a clean demonstration of QoS-aware multiplexing, until the uplink transport block was filled by the UE rather than the scheduler, as TS 38.321 requires. The baselines then reach 100/90/92% unaided; the protection was the UE's prioritised-bit-rate configuration all along. We report all three because the shortcuts are natural ones, the errors were ours, and the corrections changed conclusions.
3) An adoption decision with its reasoning (Section VIII). The design wins unconditionally on two mechanisms PF structurally lacks, and loses on a knife-edge contract-count metric under GBR infeasibility—because a threshold metric rewards concentrating a shortfall and this design spreads it. We show that no knob recovers the difference, that a scheduler-level fallback to PF would surrender the mechanisms that work, and that the strategic tier already computes an exact feasibility test making graceful degradation possible without regime guessing.

We also report two negative results—an adaptive dual-ascent GBR penalty and a spectral-efficiency tilt, both implemented, measured and rejected—and release the simulator, the scheduler library and one script per result table.

II. Background and Related Work

A. Channel-opportunistic scheduling

Round Robin is channel-blind and wastes capacity on faded users; Max-C/I maximises cell throughput and starves the cell edge; PF [1] interpolates via $r_{\text{inst}}/R_{\text{avg}}$ and is the practical default. All three share one limitation: none can be told "this flow needs 8 Mbps" or "this packet dies in 10 ms."

B. QoS-aware single-tier schedulers

M-LWDF [3] multiplies the PF metric by head-of-line delay and a weight; the Exp-rule [4] uses an exponential urgency term. These handle delay well. For rate contracts they share a subtler defect: a multiplicative urgency metric of the form $(r_{\text{inst}}/R_{\text{avg}})\cdot(1+w\cdot\text{deficit})$ has an equilibrium that equalises a weighted $1/R_{\text{avg}}$ across flows. It cannot drive delivered rate to an arbitrary, unequal target vector however large w is. This is not only an analytical objection. Monghal et al. [12] weight their frequency-domain metric by $\max(1, \text{TBR}/R_n)$ for users below a target bit rate and report that the weighting "is not efficient enough to allow 95% of the users to reach their TBR"—the saturation, measured, in the setting it was designed for. This saturation argument is why our Tier-2 is a drift-plus-penalty integrator rather than a tuned PF variant; we include a class-aware "Gradient" baseline of exactly this shape to make the difference measurable.

C. Optimisation-based and hierarchical RRM

NUM [2] states allocation declaratively as $\max \sum_i U_i(r_i)$ under capacity constraints, but yields a rate vector, whereas a wireless scheduler must decide which UE to serve this slot over a stochastic channel. Drift-plus-penalty control [6] and greedy primal-dual scheduling [8] address the per-slot problem but need a target to track. The natural composition—a slow declarative program feeding a fast tracker—is the pattern behind NVS [9], the O-RAN non-RT/near-RT RIC split [10], and two-timescale IIoT resource allocation that likewise places Lyapunov control on the slow scale and a fast solver beneath it [13]. The pattern is not our contribution; its concrete instantiation against a symbol-accurate NR resource model, and the empirical characterisation of when it pays, are.

D. Learning-based schedulers

Most recent work in this space is learned rather than declarative. xSlice places an online deep-RL agent on the near-RT RIC and adapts MAC-layer allocation against a combined throughput/latency/reliability objective [14], evaluated—unlike this paper—on a real O-RAN testbed. We deliberately do not take that route, and it is worth saying why. Kela et al. [15], arguing from inside that literature, find that deployable deep schedulers do not yet exist, citing real-time execution cost, 3GPP compliance and poor generalisation across bandwidth, MIMO and traffic configurations. For a deployment that must be commissioned and certified by plant engineers, a convex program with an auditable objective and a single fairness dial is a defensible alternative—and its failure modes, as Section IV-C shows, can be characterised analytically rather than only measured.

E. Industrial and private 5G

Closest to our target is FLEX [16], which is also QoS-aware, also industrial, and also simulator-based, but varies the TDD pattern—adjusting the UL/DL split in flexible slots—where we hold the pattern static and allocate within a fixed per-direction budget. The two are complementary. On configured grants, Larrañaga et al. [11] contribute an independent open-source CG implementation for ns-3/5G-LENA and an Industry 4.0 scheduling study; their finding that CG policy dominates latency for time-critical industrial traffic corroborates our Section VI-B from a different simulator. Our contribution there is accordingly not that CG helps, but why it helps twice over—bypassing the PDCCH budget and the BSR round-trip simultaneously—and the self-gating rule that keeps it from hurting when reservations cannot be honoured.

F. The OAI baseline

OAI's [17] NR MAC schedules with a PF metric over per-UE grants. Our ProportionalFair baseline is modelled on it, and all schedulers we compare grant per UE with one DCI each, so the comparison isolates QoS-awareness rather than grant granularity. OAI is also the intended integration target; open private-5G O-RAN testbeds built on it are now well established [18].

III. System Model

We consider a single NR cell with a static TDD pattern. Time is slotted; each slot offers a set of physical resource blocks (PRBs) in one direction, with the special slot contributing partial DL and UL symbol counts. Let $d(i) \in \{\mathrm{DL}, \mathrm{UL}\}$ be flow i's direction and C_d the capacity of direction d in PRB-symbols per second over the TDD cycle. Per-UE SNR evolves as an AR(1) process in dB and maps through a link-adaptation staircase to SE_i, bits per PRB-symbol, discounted by target BLER.

Each flow is uni-directional and carries a 5G QoS profile [19]: a class $c(i) \in \{\mathrm{GBR}, \mathrm{Delay}, \mathrm{PF}\}$, a guaranteed flow bit rate G_i (GBR only), a packet delay budget PDB_i, a 5QI-style priority_level, and an optional slice identifier. D_i denotes offered demand.

Two control-plane budgets are modelled explicitly because both bind in practice. First, a per-slot PDCCH/CCE budget: every dynamic grant costs one DCI at an aggregation level that grows as SNR falls. Second, the uplink buffer-status-report round-trip: a real gNB learns of UL data only via a delayed BSR, so the scheduler sees a lagged view of UL backlog. Semi-persistent scheduling (SPS, "Configured Grants" [20]) bypasses both—a CG user transmits on a standing allocation with no DCI and no BSR. Section VI-B shows this double bypass is decisive.

IV. Two-Tier Scheduler Design

A. Tier-1: the strategic solve

Tier-1 answers, every $\sim$1 s, what rate should each flow target? Decision variables are target rates $r_i \geq 0$ (bps) with shortfall slacks $s_i, s_\sigma \geq 0$ for GBR and slice floors. All solves share one feasible set, parameterised by a per-flow floor:

$$\mathcal{F}(\mathrm{floor}): \quad \sum_{i:d(i)=d} r_i/\mathrm{SE}_i \leq C_d \qquad \forall d \tag{1}$$

$$r_i \leq D_i \qquad \forall i \tag{2}$$

$$r_i + s_i \geq G_i \qquad \forall i\colon c(i){=}\mathrm{GBR} \tag{3}$$

$$r_i \geq \mathrm{floor}_i \qquad \forall i\colon c(i){=}\mathrm{GBR} \tag{4}$$

$$\sum_{i\in\sigma} r_i/\mathrm{SE}_i + s_\sigma \geq \phi_\sigma C_d \qquad \forall \sigma \tag{5}$$

Constraint (1) is where symbol-accurate accounting enters: r_i/SE_i is the resource a target rate consumes, and C_d counts special-slot symbols. Slice floors (5) are capped at the slice's own demand and kept soft, so an idle slice holds nothing and a busy slice borrows freely—the allocation remains work-conserving.

B. Shortfall before utility: an ordering, not a weight

Ordering contracts ahead of utility is not new: Monghal et al. [12] give below-target users absolute priority over the rest, and Zaki et al. [21] give GBR bearers strict priority over non-GBR. Both express it as a priority partition over a metric scheduler, which says who is served first but not how a shortfall is divided among them. An optimisation formulation must answer that second question too—and Section IV-C is about the answer it gives.

The conventional formulation is a single objective,

$$\max \ \textstyle\sum_i w_{c(i)} \log(r_i + \epsilon) \ - \ \sum_i p_i s_i, \tag{6}$$

with p_i "large" so that GBR floors are effectively hard whenever feasible. On our factory workload we measure the two terms at 718 and 2.4×10^{10} respectively: a ratio of 3.3×10^7. That is not a weighting. It is a lexicographic ordering expressed as a magnitude, and it has two consequences.

The first is numerical. Posed as (6) the program is badly conditioned: on a three-flow instance small enough to solve in closed form, two interior-point solvers both returned optimal_inaccurate, disagreed by 3.6× on one flow's rate, and under-served the highest-weighted class by 28% against the analytic optimum. Rescaling does not fix it—a ratio between objective terms is a property of the model, not the units. Stating the order directly does. Over $\mathcal{F}$ we solve

$$\text{B1:} \quad S^\star = \min \ \textstyle\sum_i p_i s_i + p_\sigma \sum_\sigma \mathrm{SE}_\sigma s_\sigma \tag{7}$$

$$\text{B2:} \quad \max \ \textstyle\sum_i w_{c(i)} \log(r_i + \epsilon) \ \text{ s.t. } (7) \leq S^\star \tag{8}$$

after which both solvers agree to within 100 bps of the analytic optimum. A useful side effect: p_i's absolute magnitude becomes irrelevant and only the ratios across flows matter. Both slacks are converted to bits per second before being weighed—a slice slack is natively in PRB-symbols—so that p_i and p_σ are quoted in one currency. Compared raw, the crossover between them sits at $p_i \cdot \mathrm{SE}_i$, which would make slice-versus-GBR priority a function of the radio channel rather than of policy.

C. Soft GBR floors are a knapsack

The second consequence is structural. Once the penalty dominates, (7) is $\min \sum_i (G_i - r_i)$ subject to $\sum_i r_i/\mathrm{SE}_i \leq C$: a fractional knapsack, whose optimum is greedy in spectral efficiency and therefore a vertex of the feasible polytope. Flows are served in full or abandoned outright. The solved targets are exactly that staircase: 100% for the two highest-SE flows and the four next, 50–67% for a single boundary tier, and 0% for the lowest-SE flow. Note it orders by SE, not SNR—ue2 (18 dB) and ue4 (16 dB) share an MCS step and receive identical targets.

Two controls confirm the mechanism. Sweeping p over six decades leaves the targets identical—the solution is

fixed by the knapsack structure, not the penalty magnitude, so no choice of p is a tuning opportunity. And dropping the shortfall term entirely, the worst-SE flow receives 23% of its contract rather than 0: the utility alone is SE-favouring but does not abandon, and it is the shortfall term that converts a merely SE-favouring allocation into a vertex one.

This is a property of our formulation, not of QoS-aware scheduling, and we checked: none of the canonical GBR schedulers uses a shortfall penalty. The literature encodes the contract either in a per-flow metric under a priority partition [12], [21], [22] or in the shape of the utility, kept concave precisely so the allocation problem has a unique solution [23]—which cannot exhibit this. We keep the penalty form because the lexicographic pair of Section IV-B expresses "contracts first" exactly where a concave utility only approximates it, but a reader adopting it should know what it carries.

D. A max-min GBR floor

We therefore prepend a max-min satisfaction stage. Let $\hat{G}_i = \min(G_i, D_i)$ be flow i's reachable contract; the demand cap matters, or an under-offered GBR flow pins the level at its own unreachable ratio and drags the set down. Stage A solves

$$\begin{aligned} t^\star = \max t \quad \text{s.t.} \quad & r \in \mathcal{F}(0), \;\; t \in [0,1], \\ & r_i \geq t\hat{G}_i \;\; \forall i\colon c(i)\text{=GBR} \end{aligned} \tag{9}$$

and sets $\text{floor}_i = \alpha\, t^\star \hat{G}_i$ in (4). $t^\star$ is the largest fraction of contract that every GBR flow can hold simultaneously; $t^\star = 1$ means the GBR set is jointly feasible.

This is enabled by default, and the reason it is safe is that it self-disables: whenever $t^\star = 1$ the floor binds nothing and the result is identical to omitting the stage. It costs something only in genuine GBR overload. α trades the guarantee against throughput, with $\alpha = 0$ recovering the single-stage behaviour exactly.

All programs are posed in normalised units—rates as multiples of the largest contract, capacity usage as a fraction of each direction's budget—so every coefficient is $O(1)$. This is not cosmetic: posed in raw bps, a unit-scale t against 10^7-scale rates made the solver report optimal on a $t^\star$ that was non-monotone in capacity, which is impossible for a max-min level.

E. Tier-2: drift-plus-penalty rate tracking

Each flow carries a virtual queue Q_i, a control accumulator in bits rather than a buffer of data. Per slot of duration Δ: grow $Q_i \mathrel{+}= r_i\Delta$; clamp $Q_i \leftarrow \min(Q_i, \text{ceil}_i)$; serve; drain $Q_i \leftarrow \max(0, Q_i - \text{delivered}_i)$. Because Q_i is a non-saturating integrator it can enforce an arbitrary target vector, which is precisely what the multiplicative metrics of Section II cannot do.

The clamp is subtle. Q_i is bounded by what the flow legitimately should have delivered over the trailing Tier-1 window W but did not: $\text{ceil}_i = \max(0, \min(r_i W, A_i^W) - D_i^W)$ with A^W, D^W bits arrived and delivered in the window. Using a windowed arrival count rather than instantaneous backlog is essential: a bursty video flow's buffer momentarily empties between frames, and clamping to that near-zero backlog erases its legitimate rate-tracking debt, letting continuous flows starve bursty GBR ones. This was a real regression in our implementation.

F. Per-UE grants and the MAC multiplexer

The 5G MAC grants per UE—one DCI, one transport block—so Tier-2 does too. For Delay flows a head-of-line urgency bonus enters the queue before ranking, $\tilde{Q}_i = Q_i + w_{\text{d}}(\text{HoL}_i/\text{PDB}_i)^\kappa \max_j Q_j$, scaled by the system-wide maximum queue rather than the flow's own small target—otherwise a low-rate control flow could never out-compete bulk. UEs are then ranked by $M_u = \big(\sum_{i\in u} \tilde{Q}_i\big) \cdot \text{SE}_u$, which is simultaneously opportunistic and rate-tracking, and served greedily subject to PRB and CCE budgets. A granted UE's transport block is filled across its flows by logical-channel prioritisation [20]: by priority_level, then by $\tilde{Q}_i$. The baselines use the same multiplexer (with plain backlog as the tiebreak, since they carry no virtual queues), so all schedulers are strictly comparable on DCI cost.

G. Configured grants

Periodic flows receive standing allocations sized to their contracted floor, granted in priority tiers with proportional scale-back so that no flow is dropped for being late in an enumeration order. A viability floor governs the mechanism: if a tier's reservations would be scaled below a threshold, the tier runs dynamically instead—unless dropping it would overrun the CCE budget, in which case SPS's zero-DCI property makes it the lesser evil. SPS thus self-gates, engaging where it helps and standing aside where fixed reservations would starve the dynamic pool.

V. Evaluation Methodology

A. Simulator and fidelity discipline

We evaluate in a purpose-built discrete-event simulator whose job is comparative questions, not absolute ones. The design principle is to model, to good accuracy, every resource the scheduler competes for, and nothing else. Modelled: the slot-by-slot PRB grid including the TDD special slot; the per-slot PDCCH/CCE budget with variable DCI aggregation level; the UL BSR round-trip as a per-flow delay plus Bernoulli loss, bypassed by configured grants; CQI report delay and loss, with BLER computed from the mismatch between the MCS the scheduler picked and the true SNR at transmission; per-UE AR(1) SNR through an MCS staircase; per-flow buffers with head-of-line timestamps and PDB expiry. Not modelled: LDPC and I/Q, HARQ as a full state machine (a fixed delay plus BLER discount instead), per-packet RLC segmentation, MU-MIMO, mobility and inter-cell interference.

The discipline cuts both ways, and the PDCCH budget is the case in point: modelling it is what reveals configured grants to be decisive (Section VI-B), and omitting it would have led to the opposite conclusion. The same held for the BSR round-trip, which was our largest fidelity gap until we closed it; omitting it understates SPS.

B. Scenarios

Three workloads isolate three bottlenecks. factory_robots: 10 mobile robots, 24 flows—uplink camera and LIDAR (GBR), downlink motion-control (Delay), best-effort uploads and a TCP flow—with an SNR spread of 14–24 dB and uplink demand roughly twice carrier capacity as shipped. Carrier: 40 MHz, numerology $\mu = 2$, DSUUU TDD (uplink-heavy). sensor_dense: 30 small periodic uplink sensors with a 15 ms PDB, sized so the control channel binds before the data channel. Carrier: 30 MHz, $\mu = 1$, DSUUU. latency_bound: 8 interactive 5 Mbps streams with a 12 ms PDB sharing a saturated downlink with 80 Mbps of bulk. Carrier: 40 MHz, $\mu = 1$, DDDSU (downlink-heavy).

The MCS staircase is a fitted 0.75× Shannon curve capped at 7.5 bits/RE (≈5.8 bps/Hz effective) at 31 dB. A typical factory UE at 22 dB sits at 4.5 bits/RE; a cell-edge UE at 14 dB at 2.0. This is deliberately approximate—good enough for the comparative claims here; a 3GPP TBS extract is future work.

The load sweep varies offered load around the shipped operating point. Because capacity in a real deployment is fixed by spectrum, we report the axis as load relative to the shipped point, where higher is worse. Uplink runs with an 8-slot (≈4 ms) BSR round-trip and an 8-slot CQI report delay throughout.

C. Metrics

We report contract-oriented metrics, and this is a deliberate methodological choice: mean delivery ratio hid every finding in this study. A GBR flow's contract is its GFBR, counted as met at $\geq 95\%$; a Delay flow's is $\geq 99\%$ on-time delivery within PDB. We report the count of flows meeting their contract, the minimum delivery across the GBR set, and p99 head-of-line delay. Mean and total throughput appear as context, not as the verdict—a scheduler can post a healthy 86% mean while the missing 14% is one starved safety-critical flow.

VI. Results

A. The value of QoS-awareness is a hump

Table I sweeps offered load. At the shipped load GBR demand far exceeds capacity and essentially no scheduler honours a contract; at one third of it everyone has slack and all converge. In between, the two designs differ in which metric they favour rather than in which is better. At 0.50× load the two-tier design honours 10/10 contracts against PF's 8/10; at 0.67× PF meets 6/10 to our 1/10 while its worst-served flow sits at 21% against our 67%. Section VIII takes that trade apart.

TABLE I
Load sweep on factory_robots. Load is relative to the shipped operating point; higher is worse. "met" counts GBR flows delivering $\geq 95\%$ of GFBR. −mm pins the max-min stage off.

Load	Scheduler	Total	met	mean	min
1.00×	PF	69.3 M	1/10	51%	1%
	TwoTier	65.9 M	0/10	49%	44%
	TwoTier −mm	73.1 M	1/10	57%	0%
0.67×	PF	96.5 M	6/10	69%	21%
	TwoTier	93.2 M	1/10	71%	67%
	TwoTier −mm	95.1 M	2/10	74%	47%
0.50×	PF	116.8 M	8/10	80%	52%
	TwoTier	119.4 M	10/10	83%	81%
0.33×	PF	124.8 M	10/10	85%	83%
	TwoTier	125.5 M	10/10	85%	83%

TABLE II
sensor_dense: 30 periodic uplink sensors, 15 ms PDB. On-time requires $\geq 99\%$ delivery within PDB.

Scheduler	Total	on-time	min deliv.	p99
RoundRobin	6.9 M	0/30	71%	15.0 ms
PF	9.2 M	2/30	85%	15.0 ms
TwoTier	9.6 M	30/30	100%	5.0 ms

The 0.67× row is where the metric becomes the discussion. The two-tier design meets fewer knife-edge contracts than PF (0/10 against 4/10) while delivering a strictly better distribution: minimum 60% against 4%, mean 65% against 55%. A 95% threshold rewards a scheduler that abandons some flows so others clear the bar. For a factory where every robot matters, "all robots degraded gracefully" beats "four robots perfect, six offline"—so the contract count is necessary but not sufficient, and must be read alongside the minimum.

Engineering implication. Since capacity is fixed by spectrum, the operator's lever is admission control: keep peak offered load in the moderate band, because that is where the scheduler pays for itself. A cell that systematically runs at $\gtrsim$2.5× its capacity has a capacity-planning problem that no MAC fixes.

B. Control-channel-limited: a capability PF lacks

Table II reports sensor_dense, where the DCI/CCE budget binds before the data channel. The two-tier design meets 30/30 deadlines with a 5 ms p99 tail; PF meets 2/30 and pins at the 15 ms PDB. This is not a tuning difference. Each periodic flow receives a standing allocation that costs zero PDCCH per slot and needs no BSR round-trip; PF is throttled by both budgets at once and has no mechanism to bypass either.

TABLE III
latency_bound: 8 interactive streams (12 ms PDB) versus 80 Mbps of bulk on a saturated downlink.

Scheduler	on-time	mean	p99	bulk DL
RoundRobin	3/8	84%	12.0 ms	22.8 M
PF	5/8	86%	12.0 ms	24.6 M
TwoTier	8/8	100%	4.5 ms	13.2 M

TABLE IV
Per-flow delivery as a fraction of GFBR, factory_robots at the shipped load, worst channel first. −mm pins the max-min stage off.

Flow	SNR	RR	PF	TwoTier −mm	TwoTier
ue7	14 dB	23%	28%	0%	54%
ue4	16 dB	51%	63%	0%	53%
ue9	17 dB	90%	83%	55%	55%
ue2	18 dB	57%	68%	53%	54%
ue6	19 dB	37%	46%	50%	55%
ue3	20 dB	63%	74%	69%	54%
ue10	20 dB	92%	1%	69%	59%
ue8	21 dB	100%	100%	69%	89%
ue1	22 dB	77%	92%	71%	56%
ue5	24 dB	56%	68%	80%	59%
mean		65%	62%	59%	59%

We regard this as the strongest practical result in the paper: for any deployment with dense periodic small-payload traffic—sensors, PLCs, AGV telemetry—configured-grant support is not an optimisation but a prerequisite, and capacity planning that counts only PRBs will mis-size the cell.

C. Deadline-blindness is silent

Table III reports latency_bound. The two-tier design meets 8/8 deadlines; PF meets 5/8. The trade is explicit and deliberate: bulk throughput falls from 24.6 Mbps to 13.2 Mbps to fund the interactive set.

The dangerous property is that PF's failure is quiet. Its control-flow mean delivery is 86%—a number that reads as healthy on a dashboard—but the missing 14% are aged-out packets, and in a teleoperation or motion-control loop those are precisely the safety-relevant ones. Mean delivery is not a safety metric.

D. Cell-edge starvation and the max-min floor

Table IV gives the per-flow picture at the shipped load and makes the knapsack result of Section IV-C concrete. Without the max-min stage the two lowest-SE flows land at exactly 0%—not degraded, abandoned—while high-SNR peers run at 71–80%. With the stage enabled no GBR flow falls below 53%, against PF's worst at 1%. Note the aggregate cost: PF and RR both carry a higher mean here, and the case for the design at this load is entirely distributional.

The cost is stated plainly: at the shipped load PF carries more total throughput than the two-tier design (69.3 vs 66.7 Mbps). The max-min floor gives up 4% of aggregate capacity to hold the worst-served flow at 40% of contract rather than 0%. Whether that is the right trade is a deployment question—a robot whose video is switched off entirely is a failure in a way that a uniformly degraded fleet is not—but it is a trade, and we do not present it as a free win.

Two further effects are visible. The mixed-flow UEs (ue8–ue10, carrying a GBR flow and a best-effort flow) collapse to 0–3% under the QoS-blind baselines, because their MAC multiplexer fills the transport block by raw backlog and a continuously backlogged best-effort flow wins every time. And the bill for the max-min floor is paid by the high-SNR flows: ue1 drops from 91% under PF to 56%.

E. Sensitivity to control-plane fidelity

Because our BSR and CQI models are approximations, we sweep them. BSR delay matters and is close to linear: over 0–16 slots PF's minimum delivery falls 66%→30% and its contract count breaks 8/10→5/10, while the two-tier design moves 80%→77% and 10/10→8/10 and loses about 1% of throughput against PF's 10%. SPS-served flows are invariant to BSR by construction. BSR loss is near-inert on this workload (0–20% loss moves nothing) because continuously backlogged video keeps buffers non-empty, so a lost report rarely changes the eligibility bit.

CQI staleness is likewise small here, and the reason is a property of the channel rather than of the scheduler: our AR(1) model produces per-slot SNR innovations well below the ∼3 dB spacing of MCS thresholds, so even a stale report usually picks the right MCS. A conservative semi-persistent MCS for SPS—a mobility hedge—is neutral at 0 dB margin and actively harmful beyond 1 dB, because larger reservations cross the SPS viability floor. We report this as a negative result for our deployment class: the hedge is worth paying for under mobility, and a static factory is exactly where it is not.

F. Two negative results

We implemented and rejected two mechanisms, both aimed at cell-edge starvation. An adaptive dual-ascent penalty escalates p_i on whichever flow is actually missing. It raises minimum delivery (0%→34%) but meets 0/10 contracts in both overloaded rows, because dual ascent drives toward equal normalised shortfall while a GBR contract is a step function—equalising shortfall parks every flow just below its floor so none clears it. A spectral-efficiency tilt $p_i \propto \mathrm{SE}_i^k$ with $k < 0$ rescues the cell edge only by relocating starvation to the next-worst tier.

Section IV-C explains why neither could have worked: both reweight a linear penalty whose optimum is a vertex, so they select a different victim rather than removing the abandonment. We think this is the transferable lesson—in

an infeasible allocation problem, a weight chooses whom to sacrifice; only a constraint prevents sacrifice—and it applies to any scheduler enforcing rate contracts through penalty terms.

VII. Three Modelling Shortcuts That Changed Our Conclusions

Every result in Section VI is a claim about a scheduler mediated by a simulator. Three times during this work a natural-looking modelling shortcut produced a result that did not survive being corrected. We report them with the corrected measurement beside each, because the shortcuts are ones any scheduler evaluation might make, and because in one case the artifact was a headline finding of an earlier draft of this paper.

A. The scheduler filled the uplink transport block

Our multiplexer was direction-agnostic: the same code split a UE's transport block in both directions, ordering flows by (priority, $-\tilde{Q}$) with $\tilde{Q}$ the gNB's virtual queue. In the uplink no gNB may do that. It grants a block; the UE fills it by its own logical-channel prioritisation (TS 38.321 §5.4.3.1)—two rounds in priority order, the first bounded by each channel's prioritised-bit-rate (PBR) token bucket. The gNB configures those PBRs but does not run the algorithm and never learns the split.

The shortcut handed our scheduler a per-slot lever on the uplink byte split that does not exist, and it produced a result we believed:

mixed-flow UE	RR	PF	Gradient	TwoTier
scheduler fills TB	3%	3%	3%	53%
UE fills TB	100%	100%	72%	89%

Under the shortcut the QoS-blind baselines collapse a GBR video flow to ∼3% of contract when a greedy best-effort flow shares its UE, while our design holds it near 53%. We described that as a clean demonstration of QoS-aware multiplexing. It was not: with the UE filling its own block, the baselines reach 100% unaided. The protection was the UE's PBR configuration—available to every scheduler, and nothing to do with ours. Correcting it also raised PF's contract count at 0.50× load from 5/10 to 8/10, which is most of the margin an earlier draft claimed.

B. Flows shared a priority level

With every flow at one priority, the multiplexer's priority sort is a stable sort over equal keys and silently falls back to the order flows appear in the scenario file. Results were reproducible but only accidentally: reordering a configuration file would have changed them. Deriving each flow's priority from its standardised 5QI (TS 23.501 Table 5.7.4-1) fixed it. Notably the numbers did not change—the stable sort had been producing the 5QI ordering by luck—which is precisely why this class of error survives testing.

C. Tier-1 was handed the true offered load

Our strategic tier read each flow's demand from the traffic generator's own parameters: exact, noise-free, and static. No gNB has that; it must infer demand from what it observes. Replacing the oracle with an estimator built from arrivals produced a starvation lock-in—a flow that lost out appeared to be asking for less, so its cap fell, so it lost more. Six contending flows went from $(3.9 \ldots 7.0)$ Mbps to $(0.2 \ldots 11.8)$, Jain's index $0.97 \to 0.47$.

The cause was subtle and worth naming: our estimator counted delivered + backlog, omitting bytes the flow discarded on PDB expiry. A starved flow's data ages out, so it stops appearing in the arrival count while its true offered rate is unchanged. The flow never stopped asking; we stopped counting.

The resolution was better than a fixed estimator. Tier-2's windowed ceiling already clamps a flow's virtual queue to what actually arrived, so a quiet flow is never scheduled however generous its target—and it does this from observation where the Tier-1 cap needed a prediction. The two were redundant, and the redundant copy was the one sitting where the information is worst. Removing the demand cap reproduces oracle performance exactly (32.4 M, Jain 0.97) with no demand estimate at all.

D. What we take from this

Each of the three gave the scheduler information or authority the 3GPP split does not grant it, and each flattered the scheduler rather than the baselines. That direction is not coincidence: a simulator is written from the scheduler's point of view, so its conveniences accrue there. The check we now apply is to ask, of every quantity the scheduler reads, which network element learns this, and how.

VIII. Adoption: What We Decided and Why

Two mechanisms win unconditionally: configured grants and the deadline-aware tracker (Section VI). Both address capabilities PF lacks entirely, and—the reason we trust them—both survived every correction in Section VII untouched, because neither scenario contains a multi-flow uplink UE.

The GBR picture is mixed, and the summary "PF wins in overload" is wrong. At 0.67× load PF meets 6/10 contracts to our 1/10, but delivers a worst-served flow at 26% against our 82%, and a lower mean (83% against 89%). Every robot above 82% of its contracted rate is a better outcome than six robots perfect and the worst at 26%—unless partial delivery is worthless, which for video and telemetry it is not and for a hard control loop it is. The metric, not the scheduler, is what differs: a threshold count rewards concentrating a shortfall, and this design spreads it.

We tested whether a fallback could recover the difference. Sweeping the max-min scale from 1 to 0 moves the contract count only from 1/10 to 2/10, against PF's

6/10—so the gap is not the max-min floor but the fact that Tier-1 allocates toward targets while PF's opportunism concentrates. Recovering it would need contract selection, which is admission control.

A scheduler-level switch to PF is therefore the wrong instrument: it would surrender configured grants and deadline awareness to improve one metric on one flow class. The mechanisms are independent and only the max-min floor is regime-dependent. It is also a continuous knob with a monotone effect, and the strategic tier already computes $t^\star$—an exact feasibility test for the GBR set, not a heuristic—so degradation can be graceful and needs no regime detection. Nor is detection needed for safety: at scale = 0 the design leads PF on mean and worst-case delivery at both loaded points while keeping both structural wins.

IX. Threats to Validity

Simulation, not silicon. Results are comparative and architecture-level: they establish which scheduler wins and roughly by how much, not absolute throughput, microsecond latency, real-time feasibility or UE interop. An OAI integration is the necessary next step, and the scheduler/ library is dependency-isolated for it.

Absolute figures are soft. A single run reads GBR delivery about 5 points below steady state with ±2 point scatter; we verified over 60 consecutive windows that the scheduler-versus-scheduler gap is stable, so comparative claims hold. Spectral efficiency is a fitted staircase rather than a 3GPP TBS extract.

Scenario coverage. Three synthetic scenarios isolate three bottlenecks cleanly, but they are synthetic, single-cell, and have a fixed UE population—no churn, no mobility, no inter-cell interference. Trace-driven factory workloads would strengthen external validity considerably.

Known unmodelled effects are conservative. UL k_2 grant-to-transmission timing and HARQ retransmissions consuming PRBs are both absent, and both would widen the configured-grant advantage rather than narrow it. Given Section VII, we note that this direction is asserted from the mechanism, not measured.

A contract-dimensioning bound. Even at three times capacity about 15% of GBR video bytes expire on PDB. We verified this is scheduler-independent—a lone video flow drops identically under RR, PF and our design—because an I-frame burst is 3–4× the GFBR, making a "GFBR plus tight PDB" contract internally inconsistent. It bounds what any scheduler can achieve on such a workload.

X. Conclusion

We set out to decide whether to replace the proportional-fair scheduler our private-5G stack inherits from OpenAirInterface. The answer is yes, and the reasoning is worth more than the verdict.

Two mechanisms justify the change on their own, and neither is a tuning gain: configured grants, which meet 30/30 deadlines where PF meets 2/30 once the control channel binds, and a deadline-aware per-slot tracker, without which PF's misses are invisible in mean throughput. Both reproduce results the field has reported separately, in a simulator sharing no code with the work that first reported them. On guaranteed-bitrate contracts the picture is mixed and the honest summary is that the metric differs rather than the outcome: a threshold count rewards concentrating a shortfall, this design spreads it, and which is preferable depends on whether partial delivery has value—for video and telemetry it does, for a hard control loop it does not.

The part we expect to be most useful to others is Section VII. Three natural modelling shortcuts produced results in our own evaluation that did not survive correction, and in the worst case a headline finding disappeared entirely: what we had described as our scheduler protecting multi-flow uplink UEs turned out to be the UE's prioritised-bit-rate configuration, visible only once the uplink transport block was filled by the UE rather than by us. All three shortcuts flattered the scheduler over the baselines, which we do not think is coincidence—a simulator is written from the scheduler's point of view.

We make no novelty claim for the design itself; it composes established parts, and we have described it because the composition and its failure modes are instructive. What we can offer is a decision taken carefully, the measurements behind it, the errors found on the way, and the code to check any of it.

Artifact Availability

The simulator, the scheduler library, all scenario configurations and the scripts named below will be released publicly on acceptance, and are available to reviewers on request through the programme chairs. Every quantitative claim in this paper is regenerated by a single command: Tables I, II and III by scheduler_study.py; Table IV by maxmin_study.py; the staircase, penalty sweep and utility-only control of Section IV-C by knapsack_diagnostic.py; the sensitivity results of Section VI-E by bsr_study.py and cqi_study.py. A dated engineering log recording every finding, regression and rejected design behind this paper is included.

References

[1] A. Jalali, R. Padovani, and R. Pankaj, "Data throughput of CDMA-HDR: a high efficiency-high data rate personal communication wireless system," in Proc. IEEE Vehicular Technology Conference (VTC), 2000.

[2] F. P. Kelly, A. K. Maulloo, and D. K. H. Tan, "Rate control for communication networks: shadow prices, proportional fairness and stability," J. Oper. Res. Soc., vol. 49, no. 3, pp. 237–252, 1998.

[3] M. Andrews, K. Kumaran, K. Ramanan et al., "Providing quality of service over a shared wireless link," IEEE Commun. Mag., vol. 39, no. 2, pp. 150–154, 2001.

[4] S. Shakkottai and A. L. Stolyar, "Scheduling for multiple flows sharing a time-varying channel: the exponential rule," Amer. Math. Soc. Transl., 2002.
[5] F. Capozzi, G. Piro, L. A. Grieco, G. Boggia, and P. Camarda, "Downlink packet scheduling in LTE cellular networks: key design issues and a survey," IEEE Commun. Surveys Tuts., vol. 15, no. 2, pp. 678–700, 2013.
[6] M. J. Neely, Stochastic Network Optimization with Application to Communication and Queueing Syst. Morgan & Claypool, 2010.
[7] L. Tassiulas and A. Ephremides, "Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks," IEEE Trans. Autom. Control, vol. 37, no. 12, pp. 1936–1948, 1992.
[8] A. L. Stolyar, "Maximizing queueing network utility subject to stability: greedy primal-dual algorithm," Queueing Syst., vol. 50, no. 4, pp. 401–457, 2005.
[9] R. Kokku, R. Mahindra, H. Zhang, and S. Rangarajan, "NVS: a substrate for virtualizing wireless resources in cellular networks," IEEE/ACM Trans. Netw., vol. 20, no. 5, pp. 1333–1346, 2012.
[10] O-RAN Alliance, "O-RAN architecture description," Technical Specification, non-RT and near-RT RIC timescale separation.
[11] A. Larrañaga, M. C. Lucas-Estañ, S. Lagén et al., "An open-source implementation and validation of 5G NR configured grant for URLLC in ns-3 5G LENA: a scheduling case study in Industry 4.0 scenarios," J. Netw. Comput. Appl., vol. 215, p. 103638, 2023.
[12] G. Monghal, K. I. Pedersen, I. Z. Kovács, and P. E. Mogensen, "QoS oriented time and frequency domain packet schedulers for the UTRAN long term evolution," in Proc. IEEE Vehicular Technology Conference (VTC Spring), 2008, pp. 2532–2536.
[13] Y. He, Y. Ren, Z. Zhou, S. Mumtaz et al., "Two-timescale resource allocation for automated networks in IIoT," 2022.
[14] P. Yan, J. Lu, H. Zeng, and Y. T. Hou, "Near-real-time resource slicing for QoS optimization in 5G O-RAN using deep reinforcement learning," IEEE Trans. Netw., vol. 34, pp. 1596–1611, 2026.
[15] P. Kela, B. Liu, and A. Valcarce, "From simulation to practice: Generalizable deep reinforcement learning for cellular schedulers," 2024.
[16] L. Kleinberger, M. Gundall, and H. D. Schotten, "FLEX: Joint UL/DL and QoS-aware scheduling for dynamic TDD in industrial 5G and beyond," 2026.
[17] OpenAirInterface Software Alliance, "OpenAirInterface 5G RAN," https://gitlab.eurecom.fr/oai/openairinterface5g.
[18] D. Villa, I. Khan, F. Kaltenberger, N. Hedberg, R. S. da Silva et al., "X5G: An open, programmable, multi-vendor, end-to-end, private 5G O-RAN testbed with NVIDIA ARC and OpenAirInterface," IEEE Trans. Mobile Comput., 2024.
[19] 3GPP, "TS 23.501: System architecture for the 5G system (5GS)," Technical Specification, 5QI, QFI, PDB, GFBR.
[20] ——, "TS 38.321: NR medium access control (MAC) protocol specification," Technical Specification, logical channel prioritization, HARQ, configured grants.
[21] Y. Zaki, T. Weerawardane, C. Görg, and A. Timm-Giel, "Multi-QoS-aware fair scheduling for LTE," in Proc. IEEE Vehicular Technology Conference (VTC Spring), 2011.
[22] P. Ameigeiras, Y. Wang, J. Navarro-Ortiz, P. Mogensen, and J. M. Lopez-Soler, "3GPP QoS-based scheduling framework for LTE," EURASIP J. Wireless Commun. Netw., vol. 2016, no. 78, 2016.
[23] J. Góra, "QoS-aware resource management for LTE-Advanced relay-enhanced network," EURASIP J. Wireless Commun. Netw., vol. 2014, no. 178, 2014.